FEUP - Master in Informatics and Computing Engineering

Interactive Graphics Systems 2025-2026

Practical classes

Project: Underwater ecosystem

1.Introduction

The aim of this work is to study and apply the knowledge presented in the theoretical classes of SGI. You will be accomplishing this goal with an application developed over the next nine weeks.

This edition's theme is an aquarium / marine underwater ecosystem which includes multiple elements such as fish, minerals, plants, corals, terrain, particles, different camera configurations, as well as lighting effects. This theme provides a rich experimental environment where your team can apply all the contents taught in SGI as well as those acquired from self-study.

The primary goals are:

- Develop an interactive 3D underwater environment in three.js (other extensions can be used provided they are justified and agreed with the teacher).
- Apply computer graphics fundamentals (geometry, transformations, lighting, textures, animation, collisions).
- Incorporate GLSL shaders for underwater visual enhancements.
- Create an immersive environment where users can explore the aquarium, interact with fish, and experience realistic ocean effects.

The project description will be released in three parts, to help segment the topics. This document will be updated as each new part is released.

2. Plan

Week 1 (6 October): Cameras, Primitives, and Transformations

- Set up the project framework: in the “pw2” folder of your git repository, create an empty scene (suggestion: copy pw1 files and adapt them as necessary)
- Implement multiple camera perspectives (“free-fly”, underwater view, fixed aquarium view), supported by a 2D GUI
- Create classes that are subclasses of Object3D to represent each of the following elements: terrain segments, rocks, corals, fish, bubbles. Create basic geometric primitives as placeholders for each of those classes (spheres for bubbles, cubes/planes for rocks/terrain segments, cylinders for corals, small blocks for fish).
- Structure these elements internally as a scene graph, e.g. creating a node that groups multiple fish, another that groups corals, etc.
- Apply transformations to position corals, plants, and rocks into a basic seafloor scene.
- For the mentioned features, when viewing the scene, it should be possible to switch between the "fill" and "wireframe" polygonal mode through a control also located in the GUI.
- **Milestone**: Initial static 3D environment with camera controls.
- **Git tag**: basic-modelling
- **Keep in mind:** At this point, it is more important to have the basic internal structure and scene structure, than to try to create a lot of objects and invest in their looks (that will be for later). It should be a strong scaffold for the following parts.

Week 2 (13 October): BufferGeometry, LOD, L-Systems, scene Improvements

- Replace fish geometry by a BufferGeometry that defines a set of vertices and their connectivity to form triangles, and parameterize its constructor so that the proportions of the fish body, fins and color/texture can vary (see image below for geometry suggestion).
  Add a top/dorsal fin and two belly fins as sub-objects of the fish, to allow for animation later.
  (Check TriangleBufferGeometry and BufferGeometry demos in Moodle).

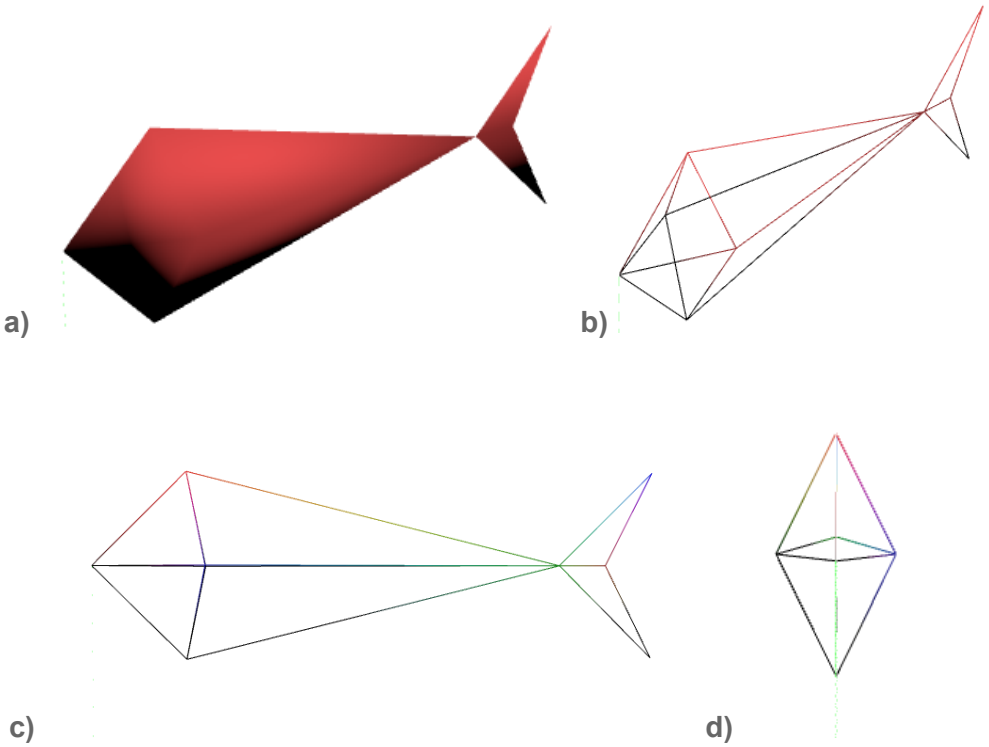

Figure 1: Suggestion for a basic fish geometry
a) Perspective shaded; b) Perspective wireframe
c) Side view; d) Front view

- Use Stochastic L-Systems (L-Systems with probabilities) to generate different corals. *(check Stochastic L-Systems in ObjectRepresentation demo in Moodle).*
- Implement a simple Level of Detail (LOD) approach for distant fish and coral to optimize rendering i.e. if they are farther than a threshold distance, use the basic blocks with the material instead of the detailed geometry.
  (Check LOD demo in Moodle)
- Improve the scenegraph hierarchy if necessary (parent/child relationships for fish groups, coral clusters, etc.).

- **Milestone**: Efficient, hierarchical scene structure with varied models.
- **Git tag**: improved-modelling

Week 3 (October 20th): Animation

- Create an animated version of the fish using a simple skeleton of two or three bones (the fish “spine” divided into two to three segments) and defining the weight for each vertex of your fish geometry, i.e. each vertex should have a set of two or three weights associated. If useful, add additional vertices/triangles to your fish mesh
  (Check Skinning 2D and 3D demos in Moodle)
- Animate the bones so that the front joint and the back joint move in a periodic left-right movement, bending the fish's body in the extremities.
  (Check Interpolation Animation demo in Moodle)
- Implement a simple keyframe animation mechanism, that given a sequence of positions underwater (a path, that can be generated randomly) and timing information, animates the position and orientation of an object. Apply that to some fish (each will have its path, some may have similar paths to move close to each other).
  (Check Keyframe Animation demo in Moodle)
- Animate seaweed and corals using sine-wave deformation to simulate underwater swaying.
  (Check Interpolation Animation demo in Moodle)
- **Milestone**: Animated underwater scene.
- **Git tag**: animation

Weeks 4, 5 (October 27th, November 3rd): Interaction

- Create a submarine that is to be controlled by the user with the keyboard
  - Simple geometry: horizontal capsule with a short vertical cylinder for the hatch (and an optional “L-shaped” tube for a periscope)
  - Keys ‘W’ and ‘S’ increase/decrease horizontal forward speed
  - Keys ‘A’ and ‘D’ change horizontal direction left or right
  - Keys ‘P’ and ‘L’ increase/decrease vertical upward speed
  - Make the “submarine” camera follow the submarine, as a first-person camera.
- Create a group of fish that uses the flock/boids approach to simulate a school of fish (“cardume”), that band together while moving around, using the Reynolds rules (https://en.wikipedia.org/wiki/Boids, check also *Flocking Demo* in Moodle)
  - rule 1: separation
  - rule 2: alignment
  - rule 3: cohesion
- Add a rule of avoidance that makes each fish move away abruptly from dangerous entities, and make the submarine one of those dangerous entities (other entities might be one or two larger keyframe-animated fish, such as sharks)
- Add simple UI elements to control flocking parameters, and camera views.
- **Milestone**: Interactive exploration of the ecosystem.
- **Git tag**: interaction

Week 6 (November 10th): Indexing techniques, intersection and collision detection

(Note: The acceleration features presented next must coexist with the so far, non-accelerated version of code)

- Integrate a Bounding Volume Hierarchy (BVH) structure that works with three.js geometry (such as https://github.com/gkjohnson/three-mesh-bvh) to efficiently index scene objects.
- Implement a 2D GUI control that allows the user to enable/disable acceleration, switching between the BVH-based and the exhaustive/brute-force implementation for the proximity checks below.
- The BVH should consider bounding boxes for the submarine, any larger keyframe-animated fish (like sharks), and the individual boids (flocking fish). You must ensure that BufferGeometries are correctly indexed by BVH
- Use the BVH structure to accelerate flocking computations by quickly identifying the relevant neighbors of a fish (both other fish, sharks, corals and submarine) - *proximity query*. This will allow each fish to calculate the necessary accelerations without iterating through *all* fish/entities in the scene.
- Integrate the BVH into an object selection process (picking), in which the user can use the mouse to select one of the entities - *intersection query*. The selected entity should be highlighted by changing its material in some way (if another entity was selected, it should be deselected).
- **Milestone**: Efficient spatial queries supporting flocking and avoidance.
- **Git tag**: acceleration

Week 7 (November 17th): Advanced textures

- Texturize objects of your choosing in the scene (e.g., corals, the submarine, fish, the terrain). In some of these cases, consider a set of Advanced Texture techniques to simulate complex material properties.
- Select at least one object and apply a texture to it, also including a bump map, to simulate fine surface irregularities like grooves or small cracks.
- Select at least one tessellated object and apply a texture to it, also including a displacement map to enhance detail and realism.
- Select an object that can be viewed at distinct distances (from near to far). Ensure textures are loaded with MIPMAPs to optimize rendering at various distances and prevent aliasing artifacts like shimmering/flickering, which can be caused by minification without proper filtering.
- Select an object that can be viewed at a steep angle and ensure the texture's filter is set to use Anisotropic Filtering to improve the visual quality of the texture when viewed at steep angles.
- Ensure the scene has at least one video texture over two objects.
- Ensure textures are small (from 128x128 to 1024x1024 or similar). Texture image size matters!
- Write a simple GLSL shader to generate and apply Perlin Noise to a specific object (fish's skin, coral or seaweed).The noise should be used to simulate a subtle, natural, and non-repeating pattern (e.g., wave distortion, swirling skin patterns, or color variation).

- **Milestone:** Improved object appearance with optimized, custom materials and correct filtering.
- **Git tag:** textures

Week 8 (November 24th): Lighting & Shadows

- Introduce appropriate lighting to simulate the underwater environment. This should include a **main light source** (e.g., a Directional Light or a faint Point Light from above) to represent the sun's rays penetrating the water. Consider using a blueish tone for this light.
- Enable and configure shadows for the main light source(s).
- Ensure that the submarine and large static objects (like the terrain, rocks, and large corals) correctly cast shadows onto the scene.
- Ensure that the terrain/floor and other static objects receive shadows cast by the movable and static elements.
- Adjust the shadow map resolution and camera settings (e.g., frustum size) of the light source to achieve a balance between shadow quality and performance.
- Add a **front light**, pointing forwards but towards the seafloor, to the user-controlled submarine using Spot Light. Make it a yellowish tone and ensure that it also casts shadows.
- Add a **flashing warning light**, with a red tone, to the periscope of the submarine.
- Change these lights' parameters so that the attenuation with distance is stronger than the default, to simulate the way light is scattered and absorbed by water.
- Add controls to the GUI for the submarine lights, for at least the front light's color and attenuation intensity, and the flashing frequency of the warning light.

- **Milestone**: Enhanced scene with light and shadow effects.
- **Git tag**: lighting

Week 9 (December 2nd): Shaders and HUD

- Use the EffectComposer to add a **Depth of Field** effect to the free-fly camera, based on the example provided in the theory class (link). Make the aperture adjustable in the UI.
- Apply a **shield effect** to the submarine, based on the example provided in the theory class (link). Make it activatable via the UI.
  Suggestion: Could be better to use a simpler proxy shape surrounding the submarine, not the submarine shape itself.
- Modify the submarine camera to represent a **periscope HUD**, inspired in figure 2, combining the following elements *through shaders*:
  - *Camera view:* the base scene rendering
  - *Depth of Field:* using the effect requested before
  - *Color Tint:* make colors more greenish/yellowish (suggestion: shader or texture)
  - *Lens Scratches/Dirt:* have some defects on the lens (texture)
  - *Crosshair:* centered on the screen (texture)
  - *Clip:* Circular cutoff of the viewport with a black border.
- Add the **submarine coordinates** as text to the periscope HUD, using **spritesheets**. The text should be updated as it moves in the scene. You can decide the reference system/origin to apply.

- **Milestone**: Visual effects and complex periscope view
- **Git tag**: shaders

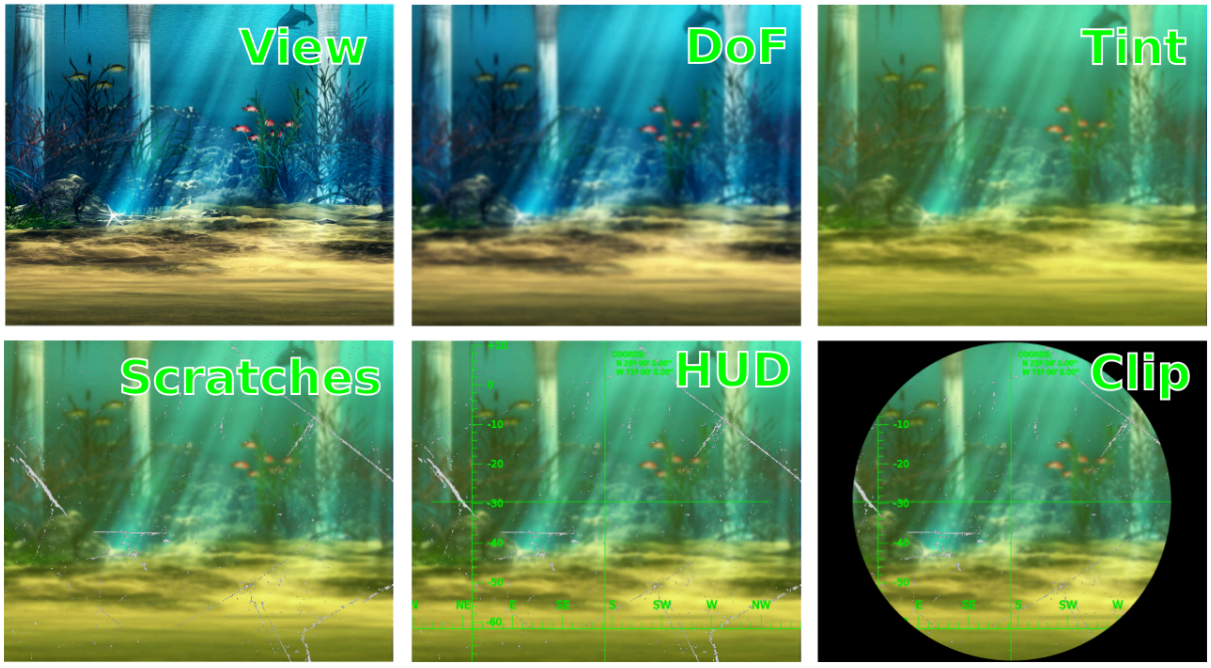

Figure 2: Suggestion for periscope HUD
Each image adds an effect to the previous image: last image (bottom-right) combines all requested effects

(The final part will be published soon; see general evaluation details in the next page)

3. Evaluation

Final Deliverables

- An interactive 3D Aquarium implemented in three.js. Make sure your application works on any computer and there are no hardwired files that work on the development environment and fail on other computers. The downloaded version of your work should be able to showcase a live demonstration containing camera views, animated ecosystem, interaction and collisions, lighting, textures, particles and performance optimizations (for instance, despite complexity, framerate is still high)
- A README.md file with:
    - Course ID
    - Class ID
    - Workgroup ID
    - Student names
    - Brief description of design choices, optimizations, shader implementations, known limitations and features not developed.
    - Describe the distribution of work among the students in the workgroup and, for each student summarize its responsibilities.

Composition of Groups: The work is carried out in groups of three students, ideally.

Group Work Assessment: The code resulting from group work will be presented and defended in the classroom, to the respective teacher. The three students in the group must be present at the assessment session (if this is impossible, they must previously agree on an alternative with the practical classes' teacher). The classification given to the three students may be different.

Students must split responsibilities evenly (for instance, modeling/geometry, animation/interaction, optimization/shaders). Transversally to the evaluation criteria presented in this document, a peer evaluation will be considered as part of the project evaluation. Lack of task ownership could cause overlaps or uneven contributions.

Use of AI tools: Although these are not prohibited, you should limit their use to help you understand things, not doing things for you. When using them, you should be critical about whatever the results from AI you depend on, and you should always be able to understand and discuss the code you submit. You don't learn by copy-pasting large swaths of code, and you lose your critical thinking abilities fast if you do so.

Assessment will focus on (details in the table below):

- The fulfilment of the functionalities defined, not forgetting:
- Controllability of the scene via the 2D interface
- The creativity put into the scene
- Developed code quality and comments
- Individual understanding of the code

Assessment will be based on weekly observations/discussions in class, code evolution, final results and presentation.

Suggested week	Content	Value
1	Cameras, Primitives, Transformations (scene setup, basic models)	1.5
2	Scene and object representation improvements	2.0
3	Animation	2.0
4	Interaction	2.0
5	Occlusion and Collision	1.5
6	Advanced Textures	2.0
7	Lighting & Shadows	1.5
8	Shaders and HUD	1.5
9	Particle Systems	1.5
	Code quality and organization (naming conventions, modular structure, commenting)	1.5
	Creativity	3.0
	Total	**20**

Main dates

- Start: 6 October 2025
- Final delivery: 19th Dec 2025